

1)

Cada processo no sistema está exatamente em um dos cinco estados seguintes, representados pelo campo "state" do descritor de processo:

- **TASK_RUNNING:** O processo é executável. Ele está atualmente em execução ou em uma fila de execução (*runqueue*) esperando para rodar. Este é o único estado possível para um processo executando no espaço de usuário.
- **TASK_INTERRUPTIBLE:** O processo está dormindo (bloqueado), esperando que alguma condição exista. Quando a condição é satisfeita, o kernel define o estado para "TASK_RUNNING". O processo também pode acordar prematuramente se receber um sinal.
- **TASK_UNINTERRUPTIBLE:** Idêntico ao anterior, exceto que não acorda ao receber um sinal. É usado quando o processo deve esperar sem interrupção ou quando o evento deve ocorrer rapidamente. Como não responde a sinais (incluindo "SIGKILL"), é usado com menos frequência.
- **__TASK_TRACED:** O processo está sendo rastreado por outro processo, como um depurador (*debugger*), via "ptrace".
- **__TASK_STOPPED:** A execução parou; o processo não está rodando nem elegível para rodar. Ocorre ao receber sinais como "SIGSTOP", "SIGTSTP", etc.

Transições:

- Um processo é criado (via "fork") e entra em **TASK_RUNNING** (pronto, mas não rodando).
- O escalonador o despacha para execução (**TASK_RUNNING** - rodando).
- Se sofrer preempção por uma tarefa de maior prioridade, volta para a fila de prontos (**TASK_RUNNING**).
- Se precisar esperar por um evento ou recurso, vai para **TASK_INTERRUPTIBLE** ou **TASK_UNINTERRUPTIBLE**. Quando o evento ocorre, volta para **TASK_RUNNING**.
- Se receber um sinal de parada ou estiver sendo depurado, vai para **STOPPED** ou **TRACED**.
- Quando termina (via "do_exit"), o processo é encerrado.

2)

No Linux, a criação de processos é feita em duas etapas distintas:

1. **fork():** Cria um processo filho que é uma cópia do pai.
2. **exec():** Carrega um novo executável no espaço de endereçamento e começa a executá-lo.

Implementação do Fork e Copy-on-Write: Tradicionalmente, o "fork" duplicaria todos os recursos do pai, o que é ineficiente. O Linux usa a técnica Copy-on-Write. Em vez de duplicar o espaço de endereçamento, pai e filho compartilham uma única cópia marcada como somente leitura. A duplicação física dos dados ocorre apenas quando um deles tenta escrever na memória. Se o "exec()" for chamado imediatamente após o "fork", os dados nunca precisam ser copiados, evitando desperdício.

Chamadas de Sistema: As chamadas "fork()", "vfork()" e "__clone()" invocam internamente a chamada de sistema "clone()" com flags específicas. O "clone()" chama a função "do_fork()", que executa a maior parte do trabalho através da função "copy_process()".

Passos do "copy_process()":

1. Chama "dup_task_struct()" para criar uma nova pilha de kernel, "thread_info" e "task_struct" idênticos ao pai.
2. Verifica limites de recursos do usuário.
3. Limpa ou reseta membros do descritor de processo (como estatísticas) para diferenciar o filho do pai.
4. Define o estado do filho como "TASK_UNINTERRUPTIBLE" para garantir que ele não rode ainda.
5. Atualiza flags (como "PF_SUPERPRIV").
6. Chama "alloc_pid()" para atribuir um PID novo.
7. Dependendo das flags passadas ao "clone()", duplica ou compartilha recursos (arquivos abertos, sistema de arquivos, tratadores de sinais, memória).
8. Retorna um ponteiro para o novo filho, que é acordado e colocado em execução (o kernel tenta rodar o filho primeiro).

3)

O Linux tem uma implementação única de threads . Para o kernel do Linux, não existe o conceito de thread. O Linux implementa todas as threads como processos padrão. Uma thread é vista simplesmente como um processo que compartilha certos recursos (como espaço de endereçamento, arquivos abertos, etc.) com outros processos. Cada thread tem seu próprio "task_struct" e parece um processo normal para o kernel. Não há lógica de escalonamento especial nem estruturas de dados separadas para threads. Isso contrasta com sistemas como Windows ou Solaris, que possuem suporte explícito a processos leves.

Threads são criadas da mesma forma que tarefas normais, usando a chamada de sistema "clone()", mas passando flags que indicam quais recursos devem ser compartilhados. Por exemplo, para criar o que é popularmente chamado de thread (compartilhando memória, sistema de arquivos, descritores de arquivos e tratadores de sinais), a chamada seria: "clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);"

Um "fork()" normal, em contraste, seria implementado como: "clone(SIGCHLD, 0);" Ou seja, o "fork" é apenas um caso especial do "clone" onde quase nada é compartilhado.

4)

Quando um processo termina (seja voluntariamente via "exit()" ou involuntariamente via sinal), a maior parte do trabalho é feita por "do_exit()". As tarefas são:

1. Define a flag "PF_EXITING" no "task_struct".
2. Chama "del_timer_sync()" para remover quaisquer timers do kernel.
3. Se a contabilidade de processos BSD estiver ativa, chama "acct_update_integrals()" para escrever informações de contabilidade.
4. Chama "exit_mm()" para liberar o "mm_struct" (espaço de endereçamento). Se ninguém mais estiver usando esse espaço, ele é destruído.
5. Chama "exit_sem()". Se o processo estiver esperando em um semáforo IPC, ele é removido da fila.
6. Chama "exit_files()" e "exit_fs()" para decrementar a contagem de uso de descritores de arquivos e dados do sistema de arquivos.
7. Define o código de saída no "task_struct" para ser recuperado pelo pai.
8. Chama "exit_notify()" para enviar sinais ao pai do processo, repassar a "paternidade" dos filhos deste processo (para outro processo no grupo de threads ou para o processo "init") e define o estado como "EXIT_ZOMBIE".

9. Chama "schedule()" para trocar para um novo processo.

Após "do_exit()", o processo é um zumbi (mantendo apenas a pilha do kernel e o "task_struct") até que o pai recolha suas informações via "wait()".